

MANAGING USERS, GROUPS AND AUTHENTICATION

OBJECTIVES

- provision multiple user accounts on multiple servers.
- Manage Linux users and groups, configure SSH, and modify Sudo configuration on managed hosts.

User Module Overview

The **user** module manages user accounts on remote hosts.

Key Capabilities

- Create, modify, or remove users
- Set home directories and shells
- Assign UIDs and groups
- Manage passwords (must be **hashed**)
- Generate SSH keys automatically

Example: Create a user and assign groups

```
``yaml
- name: Add new user to the development machine

  user:

    name: devops_user

    shell: /bin/bash

    groups: sys_admins, developers

    append: yes
``
```

Explanation:

- `name` → required; defines the username
 - `shell` → optional; sets login shell
 - `groups + append` → adds user to multiple groups without overwriting existing ones
-

Example: Generate SSH key

```
``yaml
- name: Create SSH key for user1

  user:

    name: user1

    generate_ssh_key: yes

    ssh_key_bits: 2048

    ssh_key_file: .ssh/id_my_rsa
...

```

This creates a 2048-bit RSA key pair under the user's home directory.

Common Parameters

Parameter	Description
comment	Sets account description
group	Primary group
groups	Secondary groups
home	Home directory path
create_home	Creates home directory if yes
system	Marks as system account
uid	Sets user ID
state	present or absent

Group Module Overview

The **group** module manages groups on managed hosts.

✚ Example: Ensure group exists

```
``yaml
- name: Verify that auditors group exists

  group:

    name: auditors

    state: present
...

```

⚙ Common Parameters

Parameter	Description
gid	Group ID
local	Forces local command usage
name	Group name
state	present or absent
system	Marks as system group

Quick Concept Summary

Module	Purpose	Typical Use
user	Manage user accounts	Create users, set passwords, generate SSH keys
group	Manage groups	Create or delete groups, assign GIDs

Lab Example:

- ✓ **Role 1:** Create groups (group names defined in defaults)
 - ✓ **Role 2:** Create users (user list in defaults)
 - ✓ Passwords stored in **group_vars/all/vault.yml** (NOT inside the role)
 - ✓ Users become members of groups created by Role 1
 - ✓ Users forced to change password on first login
 - ✓ SSH keys created using the user module
 - ✓ Fully reusable, environment-agnostic roles
-

ROLE 1 — create_groups

Directory Structure

roles/

└─ create_groups/

 └─ defaults/

 └─ main.yml

 └─ tasks/

 └─ main.yml

defaults/main.yml

groups_list:

- developers

- admins

- qa

tasks/main.yml

- name: Create groups from list

group:

 name: "{{ item }}"

 state: present

 loop: "{{ groups_list }}"

ROLE 2 — create_users

Directory Structure

roles/

└─ create_users/

│ └─ defaults/

│ └─ main.yml

└─ tasks/

└─ main.yml

defaults/main.yml

(Users list — no passwords here)

users_list:

- name: dev1

groups: developers

shell: /bin/bash

ssh_key_bits: 2048

ssh_key_file: ".ssh/id_rsa"

- name: admin1

groups: admins

shell: /bin/bash

ssh_key_bits: 2048

ssh_key_file: ".ssh/id_rsa"

Vault File (Stored OUTSIDE the role)

Create:

```
group_vars/all/vault.yml
```

Encrypt it:

```
ansible-vault encrypt group_vars/all/vault.yml
```

Inside:

```
...
```

```
user_passwords:
```

```
dev1: "$6$rounds=656000$ABC123$HASHEDPASSWORD"
```

```
admin1: "$6$rounds=656000$XYZ789$HASHEDPASSWORD"
```

```
...
```

✓ Passwords are SHA-512 hashed

✓ Stored outside the role (best practice)

How to generate hashed passwords

```
python3 -c "import crypt; print(crypt.crypt('MyPass123', crypt.mksalt(crypt.METHOD_SHA512)))"
```

Or you can use "mkpasswd":

➔ install mkpasswd:

```
sudo yum install whois
```

```
sudo dnf install expect
```

```
sudo dnf install mkpasswd
```

```
mkpasswd --method=sha-512
```

➔ It will prompt you for the password and output.

➔ OR use:

```
openssl passwd -6 "MyPassword123"
```

➔ OR :

```
openssl passwd -6
```

tasks/main.yml (Role 2)

This file:

- ✓ Creates users
- ✓ Assigns groups
- ✓ Sets vaulted passwords
- ✓ Forces password change
- ✓ Generates SSH keys

...

- name: Create users from list

user:

name: "{{ item.name }}"

shell: "{{ item.shell | default('/bin/bash') }}"

groups: "{{ item.groups }}"

append: yes

create_home: yes

password: "{{ user_passwords[item.name] }}"

update_password: on_create

password_expire_min: 0

password_expire_max: 0

generate_ssh_key: yes

ssh_key_bits: "{{ item.ssh_key_bits | default(2048) }}"

ssh_key_file: "{{ item.ssh_key_file | default('.ssh/id_rsa') }}"

loop: "{{ users_list }}"

...

Top-Level Playbook

...

- name: Users and Groups management

hosts: webservers

become: yes

vars_files:

- group_vars/webservers/vault.yml

roles:

- create_groups

- create_users

post_tasks:

- name: Configure sudo privileges for created users

copy:

dest: "/etc/sudoers.d/{{ item.name }}"

content: "{{ item.name }} ALL=(ALL) NOPASSWD:ALL"

owner: root

group: root

mode: '0440'

validate: "/usr/sbin/visudo -cf %s"

loop: "{{ users_list }}"

- name: Force password change on first login

command: chage -d 0 {{ item.name }}

loop: "{{ users_list }}"

Run it:

ansible-playbook site.yml --ask-vault-pass

The known_hosts Module in Ansible

1. What is the known_hosts File?

The `known_hosts` file is an SSH security file that stores **SSH host keys** of remote servers that a machine trusts. It prevents man-in-the-middle attacks by verifying the identity of servers during SSH connections.

There are two types:

- **User-specific:**
~/.ssh/known_hosts
- **System-wide:**
/etc/ssh/ssh_known_hosts

The file is created **automatically** the first time a machine uses SSH as a client — or created by Ansible when needed.

2. Why Do We Need the known_hosts Module?

SSH asks this question when connecting to a server for the first time:

Are you sure you want to continue connecting (yes/no)?

This is fine for humans, but **automation cannot type “yes”**.

In environments with many servers (10, 50, 500+):

- Cluster nodes SSH to each other
- Backup jobs use SSH
- Git pulls use SSH
- rsync uses SSH
- Scripts and cron jobs use SSH

If the host key is missing, automation **breaks**.

The known_hosts module solves this by:

- Pre-creating the `known_hosts` file
- Pre-loading trusted host keys
- Eliminating SSH prompts
- Enabling fully automated, secure SSH communication

3. What the known_hosts Module Actually Does

The module allows you to:

✓ Add a host key

Insert a server's SSH host key into the known_hosts file.

✓ Remove a host key

Remove outdated or changed keys to avoid:

WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!

✓ Manage trust relationships at scale

You can make 50 servers trust each other with one playbook.

4. How Does the Module Know Which Key to Add?

The module does **not guess** the key.

You must provide the host key, using one of these methods:

A) From a file (lookup('file'))

You store host keys on the Control Node:

pubkeys/192.168.244.192

pubkeys/192.168.244.194

Each file contains **one host key**, collected using:

```
ssh-keyscan -t rsa,ecdsa,ed25519 <host>
```

B) Dynamically using ssh-keyscan

You can let Ansible collect the key automatically:

```
key: "{{ lookup('pipe', 'ssh-keyscan ' + inventory_hostname) }}"
```

This is the modern, automatic method.

5. Example: Adding a Host Key

...

- name: Add host key for 192.168.244.192

ansible.builtin.known_hosts:

path: /etc/ssh/ssh_known_hosts

name: 192.168.244.192

key: "{{ lookup('file', 'pubkeys/192.168.244.192') }}"

...

Explanation

- path: → where to store the key
 - name: → hostname/IP to trust
 - key: → actual SSH host key (from Control Node)
-

6. Example: Adding Multiple Host Keys

...

- name: Add host keys for all nodes

ansible.builtin.known_hosts:

path: /etc/ssh/ssh_known_hosts

name: "{{ item }}"

key: "{{ lookup('file', 'pubkeys/' + item) }}"

loop:

- "192.168.244.192"

- "192.168.244.194"

...

This makes:

- **MH1 trust MH2**
- **MH2 trust MH1**
- **Both trust CN (if included)**

No SSH prompts.

Full automation.

PlayBook2:

...

- name: Automatically add SSH host keys for all hosts

hosts: all

become: yes

tasks:

- name: Add SSH host key for each host in inventory

ansible.builtin.known_hosts:

path: /etc/ssh/ssh_known_hosts

name: "{{ item }}"

key: "{{ lookup('pipe', 'ssh-keyscan ' + item) }}"

loop: "{{ groups['all'] }}"

...

THE AUTHORIZED KEY MODULE

The `authorized_key` module allows you to add or remove SSH authorized keys per user accounts. When adding and subtracting users to a large bank of servers, you need to be able to manage ssh keys.

What It Actually Does

Purpose:

Manage SSH public keys inside a user's `~/.ssh/authorized_keys` file.

This is essential when you automate user provisioning across many servers.

The module supports:

- Adding keys
- Removing keys
- Pulling keys from files
- Pulling keys from URLs (GitHub, GitLab, etc.)
- Ensuring idempotency (no duplicates)

Core Parameters

Parameter	Meaning
user	The Linux user whose authorized_keys file will be modified
key	The actual public key string
state	present (add) or absent (remove)
path	Optional custom path to authorized_keys
manage_dir	Create ~/.ssh if missing (default: yes)

Example 1 — Add a Key from a Local File

...

- name: Set authorized key for user1

authorized_key:

user: user1

state: present

key: "{{ lookup('file', '/home/user1/.ssh/id_rsa.pub') }}"

...

What happens:

Ansible reads the public key file on the *control node* and inserts it into `/home/user1/.ssh/authorized_keys` on the *managed host*.

Example 2 — Add a Key from GitHub

GitHub exposes public keys at:

`https://github.com/<username>.keys`

So you can do:

...

- name: Add GitHub public keys for user1

authorized_key:

user: user1

```
state: present

key: "{{ lookup('url', 'https://github.com/user1.keys') }}"

...
```

Why this is powerful:

If the user rotates keys on GitHub, your servers automatically stay updated.

Example 3 — Remove a Key

```
...

- name: Remove a specific key

authorized_key:

  user: user1

  state: absent

  key: "{{ lookup('file', 'old_key.pub') }}"

...
```

Example 4 — Add Multiple Keys

```
...

- name: Add multiple keys

authorized_key:

  user: deploy

  state: present

  key: "{{ item }}"

loop:

  - "{{ lookup('file', 'keys/key1.pub') }}"

  - "{{ lookup('file', 'keys/key2.pub') }}"

...
```

Lab Example:

- user1 on MH1 to SSH into MH2
- user2 on MH2 to SSH into MH1

This means each user's public key must be placed in the other machine's **authorized_keys**.

Let's break it down cleanly.

✓ Step 1 — Generate SSH keys on each managed host

On MH1 (for user1):

```
sudo -u user1 ssh-keygen -t rsa -b 4096 -N "" -f /home/user1/.ssh/id_rsa
```

On MH2 (for user2):

```
sudo -u user2 ssh-keygen -t rsa -b 4096 -N "" -f /home/user2/.ssh/id_rsa
```

Now you have:

- MH1 → /home/user1/.ssh/id_rsa.pub
 - MH2 → /home/user2/.ssh/id_rsa.pub
-

! Problem:

authorized_key **cannot read keys from remote hosts**.

It only reads from the **control node**.

So you must **collect** the public keys from MH1 and MH2 and store them on the control node.

✓ Step 2 — Copy the public keys to the control node

From CN:

```
scp user1@MH1:/home/user1/.ssh/id_rsa.pub keys/user1.pub
```

```
scp user2@MH2:/home/user2/.ssh/id_rsa.pub keys/user2.pub
```

Now CN has:

keys/user1.pub

keys/user2.pub

✅ Step 3 — Use Ansible to distribute keys

✓ user1's key must go to MH2

✓ user2's key must go to MH1

...

- hosts: MH1

tasks:

- name: Allow user2 from MH2 to SSH into MH1

authorized_key:

user: user2

state: present

key: "{{ lookup('file', 'keys/user2.pub') }}"

- hosts: MH2

tasks:

- name: Allow user1 from MH1 to SSH into MH2

authorized_key:

user: user1

state: present

key: "{{ lookup('file', 'keys/user1.pub') }}"

...

Why this works

Host	User	Needs which key?	Why
MH1	user2	user2.pub	So user2 can SSH from MH2 → MH1
MH2	user1	user1.pub	So user1 can SSH from MH1 → MH2

SSH works like this:

- The **client** uses its **private key**
- The **server** must have the **matching public key** in `authorized_keys`

So each host must store the *other* host's public key.